

Problem 15 – Smart Clinic Appointment Manager

Difficulty: ★★☆☆ Intermediate

Overview

Build a **Streamlit Clinic Appointment Manager** for a medical clinic.

The application should allow the receptionist to:

- Book appointments.
- View all appointments.
- Search for appointments.
- Cancel appointments.
- View today's schedule.
- View clinic statistics.
- Prevent appointment conflicts.

 **Streamlit only:** Do not use `input()`, `print()`, `while True`, or console menus.

1. Appointment Data

Store appointments in a **list of dictionaries**.

Each appointment should contain:

- Patient name
- Doctor name
- Appointment date
- Appointment time
- Reason for visit
- Appointment status

Initially, the appointment list is empty.

2. Application Layout

Sidebar

Display:

- Clinic name
- Current date
- Navigation/information section

The appointment data itself should be displayed in the appropriate tabs, not duplicated in the sidebar.

Tabs

Create four tabs:

1. **Book Appointment**
 2. **View Appointments**
 3. **Search**
 4. **Statistics**
-

3. Book Appointment Tab

Create an **Appointment Booking Form**.

The form must contain:

Patient Name

Use `st.text_input()`.

The name must not be empty.

Doctor

Use `st.selectbox()`.

Provide a list of clinic doctors.

Appointment Date

Use `st.date_input()`.

Appointment Time

Use `st.text_input()`.

The time should be entered in a clear format such as **10:30**.

Reason for Visit

Use `st.text_input()`.

The reason must not be empty.

Book Appointment

Use `st.form_submit_button()` as the **primary action**.

4. Appointment Validation

Before saving an appointment:

- Patient name must not be empty.
- Doctor must be selected.
- Appointment time must not be empty.
- Reason must not be empty.
- The selected doctor must not already have an appointment at the same date and time.

If information is invalid, use `st.error()`.

If a scheduling conflict is detected:

- Do not save the appointment.
- Display an appropriate `st.warning()` message.

5. Book an Appointment

After a valid booking:

1. Create the appointment record.
2. Add it to the appointment list.
3. Set its status to **Scheduled**.
4. Save the data using `st.session_state`.
5. Use `st.spinner()` to simulate processing.
6. Display `st.success()` after successful booking.

Example:

 Booking appointment...

Then:

 Appointment booked successfully!

6. View Appointments Tab

Display all scheduled appointments in an organized way.

Each appointment should show:

- Patient
- Doctor
- Date
- Time
- Reason
- Status

If there are no appointments, display an `st.info()` message.

Use appropriate formatting to make the appointment list easy to read.

7. Search Tab

Allow the receptionist to search appointments using:

- Patient name
- Doctor name
- Appointment date

Use suitable Streamlit input widgets.

Display **all matching appointments**.

If no appointment matches the search, display an appropriate `st.info()` message.

8. Cancel Appointment Tab

Allow the receptionist to cancel an existing appointment.

Collect enough information to identify the appointment, such as:

- Patient name
- Doctor
- Appointment date
- Appointment time

When cancellation is confirmed:

- Find the matching appointment.
- Change its status to **Cancelled**.
- Do not delete the appointment record.
- Display a confirmation using `st.success()`.

If the appointment cannot be found, use `st.warning()` or `st.error()`.

9. Today's Schedule

Display appointments whose date matches the **current date**.

For each appointment, display:

- Patient
- Doctor
- Time
- Reason
- Status

Cancelled appointments should not be counted as active appointments.

If there are no appointments today, display an `st.info()` message.

The current date should be obtained using Python's date functionality.

10. Statistics

Display useful clinic statistics:

- Total active appointments
- Number of appointments today
- Number of appointments for each doctor
- Doctor with the most appointments

Cancelled appointments should not be counted as active appointments.

Display the statistics clearly in the **Statistics tab**.

11. Required Text Functions

Use all six:

- `st.title()` → Main application title
- `st.header()` → Major sections
- `st.subheader()` → Smaller sections
- `st.write()` → General information and appointment details
- `st.code()` → Formatted appointment summary
- `st.divider()` → Separate major sections

The main title must be **center-aligned**.

12. Required Input Widgets

Use:

- `st.text_input()` → Patient name
- `st.text_input()` → Appointment time
- `st.text_input()` → Reason for visit
- `st.selectbox()` → Doctor
- `st.date_input()` → Appointment date
- `st.form()` → Booking form
- `st.form_submit_button()` → Book Appointment
- `st.button()` → Cancel Appointment

Use appropriate labels and icons where useful.

The **Book Appointment** button must be the **primary action**.

13. Required Status Elements

Use all five:

- `st.info()` → Information / no results
 - `st.success()` → Appointment booked or cancelled
 - `st.warning()` → Scheduling conflict / appointment issue
 - `st.error()` → Invalid information
 - `st.spinner()` → Appointment processing
-

14. Competition UI Requirements

The application must demonstrate:

Alignment

- Center the main application title.
- Keep related labels and information clearly grouped.

Colors

Use appropriate Streamlit text colors to highlight important information.

For example:

- **Green** → successful appointment/status
- **Red** → cancelled or important negative information
- **Warning colors** → scheduling conflicts

Use Streamlit's supported text-color formatting rather than unnecessary custom styling.

Buttons

- **Book Appointment** → primary button
- **Cancel Appointment** → secondary/default action

Columns

Use columns where they improve the presentation.

The Statistics tab should use organized columns for key statistics.

Use **bordered columns** where appropriate to make important information visually clear.

Icons

Use suitable icons in input fields where they improve clarity, such as:

- Patient
- Doctor
- Date
- Time

Dividers

Use `st.divider()` between major sections.

15. Appointment Summary

Display the most recently booked appointment clearly.

Use `st.code()` for a formatted summary.

The summary should include:

- Patient
- Doctor
- Date
- Time
- Reason
- Status

Example:

=====

Appointment Summary

=====

Patient : Ahmed Ali

Doctor : Dr. Sara

Date : 15/08/2026

Time : 10:30

Reason : General Checkup

Status : Scheduled

=====

16. Store Appointments

Store all appointments in a **list of dictionaries**.

Use `st.session_state` to preserve the appointment records when the application reruns.

The records must remain available when the receptionist:

- Changes tabs.
- Books another appointment.
- Searches.
- Cancels an appointment.

Cancelled appointments should remain in the records with their updated status.

17. Required Functions

Create functions for:

- **Add Appointment**
- **Find/Search Appointments**
- **Check Appointment Conflict**
- **Cancel Appointment**
- **Display Appointments**
- **Display Today's Schedule**
- **Calculate Statistics**
- **Display Appointment Summary**

You may create additional functions when needed.

Challenge Mission ★★

Weekly Clinic Management Dashboard

Expand the application into a **Weekly Clinic Management System**.

Add features to:

- Display appointments grouped by date.
 - Show each doctor's weekly schedule.
 - Track completed and upcoming appointments using today's date.
 - Display the number of appointments for each day of the week.
 - Generate a weekly clinic activity summary.
-

Python Topics

This project practices:

- Variables
- Strings
- Lists
- Dictionaries
- Integers and floats
- Type casting
- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- Membership operators
- if / elif / else
- for loops
- Indexing
- Slicing
- User-defined functions
- with / context managers where appropriate
- Date handling
- Current date
- Date components
- Basic date calculations

Streamlit Topics

Displaying Text

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.selectbox()`
- `st.date_input()`
- `st.form()`
- `st.form_submit_button()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Layout & State

- `st.sidebar`
 - `st.tabs()`
 - `st.columns()`
 - `st.session_state`
-

Final Checklist

Data

- Appointments stored as dictionaries
- Appointment status tracked
- `st.session_state` used

Interface

- Sidebar
- 4 tabs
- Booking form
- Centered title
- Appropriate colors
- Primary/secondary buttons
- Icons
- Dividers
- Organized columns

Booking

- Patient name
- Doctor
- Date
- Time
- Reason
- Validation
- Conflict detection
- Appointment confirmation

Appointment Management

- View all appointments
- Search appointments
- Cancel appointments
- Today's schedule
- Statistics
- Recent appointment summary

Required Streamlit

- All text functions
- All input widgets
- All five status elements
- Sidebar
- Tabs
- Form
- Columns
- Session state

Challenge

- Weekly appointments
- Doctor schedules
- Completed/upcoming appointments
- Daily totals
- Weekly summary

Important Restrictions

Use **Streamlit only** for user interaction.

Do not use:

- `input()`
- `print()`
- `while True`
- Console menus
- Console interaction

The application should use Streamlit widgets, functions, and `st.session_state` to manage the complete workflow.

Learning Objectives

After completing this project, you should be able to:

- Organize real-world appointment data using lists and dictionaries.
- Validate information before storing records.
- Detect scheduling conflicts.
- Search, update, and manage records.
- Work with dates and schedules.
- Use `st.session_state` to maintain application data.
- Build a multi-tab Streamlit application with forms, sidebar, columns, and status elements.
- Apply competition-style UI requirements including **colors, alignment, buttons, icons, borders, and dividers**.
- Generate useful clinic statistics from stored appointment data.